Side Activity 7: Multi-Table Joins, Views, and Triggers

CSCI3170 Introduction to Database Systems

Learning Objectives

In this Side Activity, you will learn:

- How to write **multi-table joins** using `JOIN ... ON`, `JOIN ... USING`, and `NATURAL JOIN`.
- Why **missing join conditions** create a **Cartesian product** (often a bug).
- How to use **views** to provide a clean interface over joined tables.
- How to write a **simple trigger** to fill a value automatically.

Part 0: Getting started

Please refer to the instructions in **Side Activity 3** to access your Oracle environment (SQL*Plus).

Part 1: Multi-table joins

Key Concept: A join is about matching rows

A join combines rows from tables based on a matching condition. If you do not specify the matching condition, Oracle will generate **all combinations** of rows (Cartesian product).

Step 1: Create sample tables

We use a simple university schema: students, courses, and enrollments.

```
CREATE TABLE student_t (
  student_id VARCHAR2(10) PRIMARY KEY,
  name       VARCHAR2(50) NOT NULL,
  major      VARCHAR2(30)
);

CREATE TABLE course_t (
  course_id  VARCHAR2(10) PRIMARY KEY,
  title      VARCHAR2(100) NOT NULL,
  dept       VARCHAR2(30)
);

CREATE TABLE enroll_t (
  student_id VARCHAR2(10) NOT NULL,
  course_id  VARCHAR2(10) NOT NULL,
```

```
  term       VARCHAR2(10) NOT NULL,
  CONSTRAINT enroll_pk PRIMARY KEY (student_id, course_id, term),
  CONSTRAINT enroll_student_fk FOREIGN KEY (student_id) REFERENCES student_t(
    student_id),
  CONSTRAINT enroll_course_fk  FOREIGN KEY (course_id)  REFERENCES course_t(course_id)
);

INSERT INTO student_t VALUES ('S001', 'Alice', 'CS');
INSERT INTO student_t VALUES ('S002', 'Bob',   'EE');
INSERT INTO student_t VALUES ('S003', 'Cathy', 'CS');

INSERT INTO course_t VALUES ('CSCI3170', 'Database Systems', 'CS');
INSERT INTO course_t VALUES ('CSCI3100', 'Software Eng',     'CS');
INSERT INTO course_t VALUES ('ELEG2200', 'Digital Logic',    'EE');

INSERT INTO enroll_t VALUES ('S001', 'CSCI3170', '2026S');
INSERT INTO enroll_t VALUES ('S001', 'CSCI3100', '2026S');
INSERT INTO enroll_t VALUES ('S002', 'ELEG2200', '2026S');
INSERT INTO enroll_t VALUES ('S003', 'CSCI3170', '2026S');
```

Step 2: Join with `ON` (explicit condition)

Use `JOIN ... ON` when you want full control over the matching condition.

```
SELECT e.term, s.student_id, s.name, c.course_id, c.title
FROM enroll_t e
JOIN student_t s ON s.student_id = e.student_id
JOIN course_t  c ON c.course_id  = e.course_id
ORDER BY e.term, s.student_id, c.course_id;
```

Step 3: Join with `USING` (same column name)

`USING(col)` is a shortcut when both tables have a column with the same name.

```
SELECT term, student_id, course_id
FROM enroll_t
JOIN student_t USING (student_id)
JOIN course_t  USING (course_id)
ORDER BY term, student_id, course_id;
```

Step 4: NATURAL JOIN (automatic matching columns)

`NATURAL JOIN` automatically joins on **all columns with the same name**. This can be convenient, but it can also become dangerous if schemas change (a new column name match changes the join behavior).

```
SELECT term, student_id, course_id
FROM enroll_t
NATURAL JOIN student_t
NATURAL JOIN course_t
ORDER BY term, student_id, course_id;
```

Why NATURAL JOIN can surprise you

If later you add a column with the same name to both tables, a NATURAL JOIN will start matching on it automatically and your result may change without warning.

Step 5: Missing join condition → Cartesian product (a common bug)

The following query is **wrong** because it does not specify how to match students to enrollments.

```
SELECT s.student_id, s.name, e.course_id
FROM student_t s, enroll_t e;
```

What to observe

If you run it, the result size becomes `(#students) * (#enrollments)`. This is almost never what you want.
Fix it by adding a join condition (either `WHERE s.student_id = e.student_id` or a proper `JOIN ... ON`).

Part 2: Views

Key Concept: Views as a stable interface

A view is a stored query. You can expose a view to users so they do not need to remember complex joins, and you can update the view definition without changing user queries (using `CREATE OR REPLACE VIEW`).

Step 1: Create a view for enrollments

```
CREATE OR REPLACE VIEW v_enrollment_detail AS
SELECT e.term, s.student_id, s.name, s.major, c.course_id, c.title, c.dept
FROM enroll_t e
JOIN student_t s ON s.student_id = e.student_id
JOIN course_t  c ON c.course_id  = e.course_id;
```

Step 2: Query the view

```
SELECT * FROM v_enrollment_detail
ORDER BY term, student_id, course_id;
```

Step 3: Replace the view

Suppose we only want to expose CS department courses in the view.

```
CREATE OR REPLACE VIEW v_enrollment_detail AS
SELECT e.term, s.student_id, s.name, c.course_id, c.title
FROM enroll_t e
JOIN student_t s ON s.student_id = e.student_id
JOIN course_t  c ON c.course_id  = e.course_id
```

```
WHERE c.dept = 'CS';
```

```
SELECT * FROM v_enrollment_detail
ORDER BY term, student_id, course_id;
```

Part 3: A simple trigger

Key Concept: Triggers run automatically

A trigger is code that runs automatically when a table is modified. In this Side Activity, we use a simple `BEFORE INSERT` trigger to fill a missing value.

Step 1: Create a table and a trigger

We create a log table. If `created_at` is not provided, we want Oracle to fill it with `SYSDATE`.

```
CREATE TABLE audit_log_t (
  log_id     NUMBER PRIMARY KEY,
  message    VARCHAR2(200) NOT NULL,
  created_at DATE
);

CREATE OR REPLACE TRIGGER audit_log_fill_time
BEFORE INSERT ON audit_log_t
FOR EACH ROW
BEGIN
  IF :NEW.created_at IS NULL THEN
    :NEW.created_at := SYSDATE;
  END IF;
END;
/
```

SQL*Plus note

The trailing slash `/` is required in SQL*Plus to execute the trigger definition.

Step 2: Test the trigger

```
INSERT INTO audit_log_t(log_id, message, created_at)
VALUES (1, 'explicit time', SYSDATE - 1);

INSERT INTO audit_log_t(log_id, message)
VALUES (2, 'time filled by trigger');

SELECT * FROM audit_log_t ORDER BY log_id;
```

Part 4: Self Challenge Exercises (not graded)

Task 1: Joins (USING, NATURAL JOIN, and Cartesian product)

Write SQL to satisfy all requirements below.

1. Create and populate three tables `student_x`, `course_x`, `enroll_x` with the same column meanings as the tutorial.
2. Write a 3-table join query using **JOIN ... ON** that outputs: `term, student_id, student_name, course_id, course_title`. Sort by `term, student_id, course_id`.
3. Write a query using **JOIN ... USING** (must use `USING` at least once) that outputs `term, student_id, course_id`.
4. Write a query using **NATURAL JOIN** that outputs the same columns as the USING query.
5. Demonstrate a **Cartesian product bug**:
   - Write an incorrect query that accidentally produces a Cartesian product.
   - Show how you detected it (e.g., row count is too large).
   - Provide the corrected query.

Task 2: Views

1. Create a view `v_enroll_x` that joins your three tables and exposes: `term, student_id, student_name, course_id, course_title`.
2. Query the view to show at least 5 rows.
3. Use `CREATE OR REPLACE VIEW` to change the view so it only shows CS department courses (or a similar filter rule).
4. Query the view again to show the result changed.

Task 3: Trigger

1. Create a table `audit_x(log_id PK, message NOT NULL, created_at DATE)`.
2. Create a **BEFORE INSERT FOR EACH ROW** trigger that fills `created_at` with `SYSDATE` when it is NULL.
3. Insert two rows:
   - one with an explicit `created_at`
   - one without `created_at`
4. Verify with a SELECT that the trigger worked.

Task 4: Clean up

Provide cleanup commands (DROP VIEW / DROP TABLE / DROP TRIGGER). Requirements:

- Drop views before dropping their base tables.
- Drop the trigger before dropping its table.

End of Side Activity 7.